

ATHENA-LOG

Overwatch Hero Tracking & Strategy Tool

3BHITM | Tamás Lipták | Sommerprojekt 2026

Was ist Athena-Log?

Athena-Log ist ein interaktives Strategie-Tool, das Overwatch-Spielern hilft, Hero-Counter zu verstehen, Skins zu verwalten und taktische Notizen zu führen.

Datenbankstruktur (3NF)

users

userID PK
username
passwordHash
profilepicture
mainHeroID FK

heroes

heroID PK
name
role
portrait
abilities
best_maps (JSON)

skins

skinID PK
hero_name
name
rarity
cost
source
image_url

counters

counterID PK
heroID FK
counteredByHeroID FK
severity
counterTips
goodComps (JSON)

counter_notes

noteID PK
heroID FK
userID FK
noteText
isPublic
counterTips

maps

mapID PK
name
location
gamemode
screenshot

Tabellenbeziehungen & Schlüssel

users → counter_notes

via: userID

1 User → n Notizen
Personalisierte Taktiken pro Hero

heroes → counters

via: heroID / counteredByHeroID

1 Hero → n Counter-Einträge
Bei heroID UND als Gegner-Hero

heroes → counter_notes

via: heroID

1 Hero → n öffentliche
oder private User-Notizen

heroes → skins

via: hero_name

1 Hero → n Skins
Gefiltert nach Rarity, Source

skins → users

via: (Inventar-Logik)

Skin-Besitz per
User-Session gespeichert

maps → heroes

via: best_maps (JSON)

Map ↔ Hero Affinität
als JSON Array in heroes

get_counters.php | Dynamische Counter-Abfrage

```
<?php
header('Content-Type: application/json');
$conn = new mysqli("localhost", "root", "", "athena_db");

$heroID = isset($_GET['heroID']) ? (int)$_GET['heroID'] : 0;

if ($heroID > 0) {
    $query = "
        SELECT c.*, h_counter.name AS counteredByName,
               h_counter.portrait AS counteredByPortrait
        FROM counters c
        JOIN heroes h_counter ON h_counter.heroID = c.counteredByHeroID
        WHERE c.heroID = ?
        ORDER BY c.severity DESC, h_counter.name ASC";

    $stmt = $conn->prepare($query);
    $stmt->bind_param("i", $heroID);
    $stmt->execute();
    $result = $stmt->get_result();

    $counters = [];
    while ($row = $result->fetch_assoc()) {
        // JSON-Felder dekodieren
        $row['goodComps'] = json_decode($row['goodComps'], true)
?: [];
        $row['dangerousComps'] = json_decode($row['dangerousComps'], true)
?: [];
        $counters[] = $row;
    }
    echo json_encode($counters);
}
```

Prepared Statement

Verhindert SQL-Injection durch bind_param() – heroID wird als Integer gecastet.

JOIN heroes × 2

counters joiniert heroes zweimal: einmal für den gesuchten Hero, einmal für den Counter-Hero.

JSON decode

goodComps & dangerousComps sind als JSON-Strings gespeichert und werden serverseitig dekodiert.

Severity-Sort

ORDER BY severity DESC liefert die gefährlichsten Counter zuerst.

save_note.php & vote.php | Auth + Write-Operationen

```
<?php
session_start();
header('Content-Type: application/json');

// Session-Schutz
if (!isset($_SESSION['loggedIn']) || !$_SESSION['loggedIn']) {
    echo json_encode(["error" => "Not logged in"]);
    exit;
}

$data = json_decode(file_get_contents('php://input'), true);
$heroID = isset($data['heroID']) ? (int)$data['heroID'] : 0;
$userID = (int)$_SESSION['userData']['userID'];
$counterTips = isset($data['counterTips']) ? trim($data['counterTips']) : '';

// Validierung
if (!$heroID || !$counterTips) {
    echo json_encode(["error" => "Invalid data"]);
    exit;
}

$conn = new mysqli("localhost", "root", "", "athena_db");
$stmt = $conn->prepare(
    "INSERT INTO counter_notes
    (heroID,userID,counteredByHeroID,severity,counterTips,
    teammateHelp,goodComps,dangerousComps,isPublic)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)"
);
$stmt->bind_param("iiiiissssi",
    $heroID, $userID, $counteredByHeroID, $severity,
    $counterTips, $teammateHelp, $goodComps, $dangerousComps, $isPublic
);
$stmt->execute();
echo json_encode(["success" => true]);
```

vote.php – Wöchentlich-Voting

```
ON DUPLICATE KEY UPDATE
    vote_value = VALUES(vote_value),
    voted_at = CURRENT_TIMESTAMP
```

Pro User / Woche nur 1 Vote pro Kategorie (Hero, Map, Skin) über UNIQUE Key + ON DUPLICATE KEY UPDATE.

php://input:

JSON-Body wird direkt gelesen statt \$_POST, da fetch() mit Content-Type JSON sendet.

Session-Auth:

Jeder schreibende Endpoint prüft zuerst die Session – kein Login = sofortiger Exit.

Timezone-Fix:

vote.php setzt Europe/Vienna für den Montag-Wochenstart der Voting-Perioden.

Session-Handling & Dynamische Navigation

```
<?php // Jede Seite beginnt so:
session_start();

$host = "localhost"; $user="root"; $pass=""; $db="athena_db";
$conn = new mysqli($host, $user, $pass, $db);

$loggedIn = isset($_SESSION['loggedIn'])
    && $_SESSION['loggedIn'] === true;
$userData = $loggedIn ? $_SESSION['userData'] : null;

$navUsername = $loggedIn ? $userData['username'] : 'Guest';
$navPfp = ($loggedIn && !empty($userData['profilepicture']))
    ? $userData['profilepicture']
    : './media/profilepictures/default/default.png';
?>

<!-- In der Navigation (HTML): -->
<div class="userNav">
    <a href="./account.php" class="navItem">
        <? = htmlspecialchars($navUsername) ?>
    </a>
    <div class="userPfp"
        style="background-image: url('<? = $navPfp ?>');
            background-size: cover;
            background-position: center;">

    </div>
</div>
```

File-Upload

```
// account.php
$ext = strtolower(
    pathinfo($_FILES['pfp']['name'],
        PATHINFO_EXTENSION)
);
$allowed = ['jpg', 'jpeg', 'png', 'gif', 'webp'];

if (in_array($ext, $allowed)) {
    $dest = "media/profilepictures/uploads/"
        . $_SESSION['userData']['username']
        . "_pfp." . $ext;
    move_uploaded_file(
        $_FILES['pfp']['tmp_name'], $dest
    );
}
```

Live-Demo